

Assignment 3 — Bridge Pattern

Astana IT University · School of Software Engineering · ShP-2216 Software Design Patterns

Student	Bekarys Zhassuzakh
Group	SE-2527
Topic	A — Drawing
Repository	github.com/bekaryszhasuzak2007/Assignment3SDP
Base commit	88e37a37543f89a3bc5b1ee6bf56e1f046a04506
Extension commit	7d6a5ec2a5f7b325e5388841538f6fd0f7e1326f

1. Design and Bridge rationale

The application models drawing as two independently varying dimensions: the **shape abstraction** (Shape → Circle, Square) and the **rendering implementation** (Renderer → VectorRenderer, RasterRenderer, AsciiRenderer). The assignment requires these dimensions to be connected through composition rather than through a subclass for every shape/renderer combination [1]. The Bridge Pattern fits because the abstraction stores an interface-typed Renderer reference and delegates low-level rendering operations to it. A Circle can therefore work with VectorRenderer, RasterRenderer, or AsciiRenderer without changing Circle itself.

2. UML class diagram

Abstraction hierarchy		Implementor hierarchy
Shape - id : String - renderer : Renderer + Shape(id, renderer) + getId() + setImplementation(Renderer) + execute() : String	composition renderer : Renderer ■■■■■■■■■■■■■■■■■■■■	Renderer + renderCircle(int) : String + renderSquare(int) : String
▲ Circle - radius : int + execute() : String		▲ VectorRenderer RasterRenderer AsciiRenderer
▲ Square - side : int + execute() : String		All three implement Renderer .

Figure 1. UML-style view matching the submitted code. The bridge is the interface-typed field **renderer : Renderer** stored by Shape; Circle and Square do not depend on concrete renderer classes.

3. Five annotated Clean Code excerpts

1 — Separation of responsibilities

```
protected Renderer renderer;  
public Shape(String id, Renderer renderer) {  
    this.id = id;  
    this.renderer = renderer;  
}
```

Benefit: Shape stores the bridge, while concrete renderers perform rendering. This keeps high-level shape data separate from rendering details.

2 — Meaningful names

```
public String renderCircle(int radius) {  
    return "VECTOR circle radius=" + radius;  
}
```

Benefit: Names such as renderCircle and radius communicate the operation and domain value directly, so the method is easy to understand at the call site.

3 — Small focused method

```
public String execute() {  
    return renderer.renderCircle(radius);  
}
```

Benefit: Circle.execute() has one responsibility: delegate the circle rendering operation. The workflow is short and easy to trace.

4 — Avoidance of duplicated workflow

```
public void setImplementation(Renderer renderer) {  
    this.renderer = renderer;  
}
```

Benefit: Runtime switching is implemented once in the base abstraction. Circle and Square reuse the same bridge mechanism instead of duplicating renderer-switching logic.

5 — Encapsulation of state

```
private int radius;  
  
public int getRadius() {  
    return radius;  
}
```

Benefit: Circle owns its radius as private state. The renderer receives the value through the operation instead of directly accessing Circle's field.

4. Runtime switching and required checks

The two-by-two base version was completed first and committed as the base commit. T1–T4 cover Circle/Square with VectorRenderer/RasterRenderer. T5 proves that the same Circle object can change its implementation at runtime. The object reference is checked with `==`, while the ID and radius are checked separately. The actual output changes from the vector implementation to the raster implementation, but the Circle object and its domain data remain the same [1].

T5 evidence	Actual value
sameObject	true
stateUnchanged	true
before	VECTOR circle radius=2
after	RASTER circle radius=2

5. Independent I3 extension

After the base commit, AsciiRenderer was added as I3. The existing Shape, Circle, Square, Renderer, VectorRenderer, and RasterRenderer classes were not changed for the extension. Only the new I3 class and Main's demonstration code were added/updated within `src/`, as required. The saved `extension.diff` records the source change from the base version to the I3 extension [1]. This demonstrates the main benefit of Bridge: a new implementation can be introduced without multiplying abstraction subclasses.

T1–T7 expected and observed results

Check	Combination	Result
T1	Circle + VectorRenderer	VECTOR circle radius=2
T2	Circle + RasterRenderer	RASTER circle radius=2
T3	Square + VectorRenderer	VECTOR square side=3
T4	Square + RasterRenderer	RASTER square side=3
T5	same Circle: Vector → Raster	before VECTOR; after RASTER
T6	Circle + AsciiRenderer	ASCII circle radius=2
T7	Square + AsciiRenderer	ASCII square side=3

6. Bridge vs Adapter

Bridge	Adapter
Separates an abstraction from its implementation so both can vary independently.	Converts an existing interface into another interface expected by a client.
Designed around two cooperating hierarchies connected by composition.	Usually wraps an existing class/object to make its interface compatible.
In this project: Shape uses the Renderer interface and can switch Vector/Raster/Ascii implementations.	In this project: AsciiAdapter can be used if an existing incompatible renderer is available.

7. Trade-off and conclusion

A trade-off of Bridge is additional abstraction: the solution introduces an interface and a second hierarchy, which can be more code than a single direct implementation. In return, the design avoids a class explosion such as CircleVector, CircleRaster, SquareVector, and SquareRaster, and it supports runtime implementation switching. For this drawing application, the separation makes adding AsciiRenderer independent of the existing shape hierarchy.

References

[1] Astana IT University, School of Software Engineering. *Assignment 3 | Bridge Pattern*, ShP-2216 Software Design Patterns, 2026–2027. Course assignment instructions.

[2] E. Gamma, R. Helm, R. Johnson, J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. Bridge and Adapter pattern descriptions.

Build commands used for the submission: `javac --release 17 -encoding UTF-8 -d out "@sources.txt"; java -cp out Main --demo`. The submitted demo produced SUMMARY: 7/7 PASS.